

Summary Post

In the absence of peer feedback, I conducted a critical self-assessment, simulating potential counter-arguments to enhance my initial analysis of VeraCrypt versus BitLocker. This process revealed that I had underemphasised the role of usability as a primary security control.

From this perspective, I partially revise my earlier position. While I still recommend VeraCrypt for technically competent users, I now more explicitly acknowledge the argument in favour of BitLocker: its integration with the Windows ecosystem and Trusted Platform Module (TPM) reduces configuration complexity and therefore mitigates user-induced vulnerabilities. I agree with this counter-position, as many real-world security failures stem from misconfiguration rather than weaknesses in cryptographic design.

However, I maintain my original argument that VeraCrypt provides stronger assurances in contexts where transparency and independent verification are required. Its open-source architecture aligns with secure software development principles and reduces reliance on vendor trust. I therefore disagree with the implicit assumption that usability should always take precedence over transparency; instead, the priority should be determined by the threat model and operational context.

To enhance my original analysis, I now explicitly incorporate usability as a formal risk factor within the ontology. In addition to Weakness, Trigger, and Impact, usability complexity should be treated as a contributing condition that increases the likelihood of insecure behaviour. This refinement strengthens the explanatory power of the model by more clearly linking human factors to technical risk.

This reflective process has improved my ability to critically evaluate security tools beyond their technical specifications. I have developed a more balanced perspective that integrates human behaviour, system design, and contextual risk, rather than focusing primarily on cryptographic transparency. Even without external input, constructing and evaluating counter-arguments has proven valuable in deepening my understanding of secure software selection.